The DOM Tree, Traversing Nodes, and Creating/Deleting Elements in JavaScript

These are some of the most important JavaScript concepts because they allow you to **read, modify, create, and remove HTML elements dynamically**.

Think of JavaScript as a person who can walk around a webpage, inspect elements, create new ones, and remove old ones.

---

1. What is the DOM?

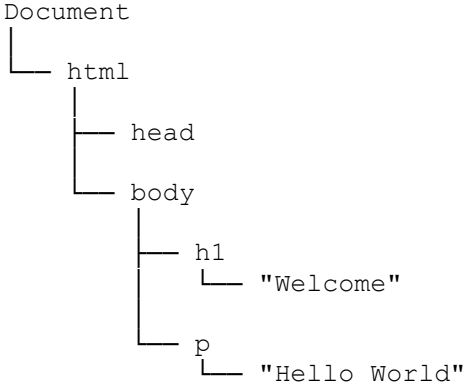
DOM stands for **Document Object Model**.

When a browser loads an HTML page, it converts the HTML into a **tree-like structure** called the DOM.

Example HTML:

```
<body>
  <h1>Welcome</h1>
  <p>Hello World</p>
</body>
```

Browser converts it into:



This structure is called the **DOM Tree**.

Every element becomes a **node**.

---

2. Types of Nodes

There are different kinds of nodes.

Element Node

```
<h1>Hello</h1>
```

`h1` is an element node.

Text Node

```
<h1>Hello</h1>
```

`Hello` is a text node.

Document Node

The entire webpage.

```
document
```

represents the root node.

---

3. Why DOM Exists

Without DOM:

HTML would be static.

With DOM:

JavaScript can:

- Change text
- Change styles
- Add elements
- Remove elements

- Handle events

Example:

```
<h1 id="title">Hello</h1>
document.getElementById("title").textContent = "Welcome";
```

Output:

```
<h1>Welcome</h1>
```

JavaScript modified the DOM.

---

4. Visualizing a DOM Tree

HTML:

```
<div id="container">
  <h1>Title</h1>
  <p>Paragraph</p>
</div>
```

DOM:

```
div
│
├── h1
│    └── "Title"
│
└── p
     └── "Paragraph"
```

JavaScript can move through this tree.

This is called **DOM Traversal**.

---

5. What is DOM Traversal?

Traversal means:

Moving from one node to another node in the DOM Tree.

Just like moving through folders on your computer.

Example:

```
div
├── h1
└── p
```

You can move:

- Parent → Child
- Child → Parent
- Sibling → Sibling

---

6. Parent Node

HTML:

```
<div id="parent">
  <p id="child">Hello</p>
</div>
```

JavaScript:

```
const child = document.getElementById("child");

console.log(child.parentNode);
```

Output:

```
<div id="parent">
```

The parent of `p` is `div`.

---

7. Child Nodes

HTML:

```
<div id="box">
  <h1>Title</h1>
  <p>Text</p>
</div>
const box = document.getElementById("box");
```

```
console.log(box.children);
```

Output:

```
HTMLCollection(2)
```

Contains:

```
<h1>
<p>
```

Access Specific Child

```
console.log(box.children[0]);
```

Output:

```
<h1>Title</h1>
```

---

8. First Child and Last Child

```
box.firstElementChild
```

Output:

```
<h1>
```

```
box.lastElementChild
```

Output:

```
<p>
```

---

9. Sibling Traversal

HTML:

```
<ul>
  <li>Apple</li>
  <li id="orange">Orange</li>
  <li>Mango</li>
</ul>
const orange = document.getElementById("orange");
```

Next Sibling

```
console.log(orange.nextElementSibling);
```

Output:

```
<li>Mango</li>
```

Previous Sibling

```
console.log(orange.previousElementSibling);
```

Output:

```
<li>Apple</li>
```

---

10. Common Traversal Properties

Property	Meaning
parentNode	Parent node
children	All child elements
firstElementChild	First child
lastElementChild	Last child
nextElementSibling	Next sibling
previousElementSibling	Previous sibling

---

11. Creating Elements Dynamically

One of the most powerful DOM features.

JavaScript can create new HTML elements.

Step 1: Create Element

```
const p = document.createElement("p");
```

Creates:

```
<p></p>
```

but not visible yet.

Step 2: Add Content

```
p.textContent = "Hello World";
```

Now:

```
<p>Hello World</p>
```

Step 3: Add to Page

```
document.body.appendChild(p);
```

Output:

```
<body>
  <p>Hello World</p>
</body>
```

---

12. appendChild()

Adds a child to the end.

```
parent.appendChild(child);
```

Example:

```
const li = document.createElement("li");

li.textContent = "Mango";

document.querySelector("ul").appendChild(li);
```

Before:

```
<ul>
  <li>Apple</li>
</ul>
```

After:

```
<ul>
  <li>Apple</li>
  <li>Mango</li>
</ul>
```

---

13. append()

Modern alternative.

```
parent.append(element);
```

Can add text too.

```
div.append("Hello");
```

---

14. Creating Multiple Elements

```
const ul = document.querySelector("ul");

for(let i = 1; i <= 3; i++) {
    const li = document.createElement("li");

    li.textContent = `Item ${i}`;

    ul.appendChild(li);
}
```

Output:

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

---

15. insertBefore()

Insert before a specific node.

```
parent.insertBefore(newNode, existingNode);
```

Example:

```
const li = document.createElement("li");

li.textContent = "Banana";

const firstItem = document.querySelector("li");

ul.insertBefore(li, firstItem);
```

Output:

```
<ul>
  <li>Banana</li>
  <li>Apple</li>
</ul>
```

---

16. insertAdjacentHTML()

Insert HTML directly.

```
div.insertAdjacentHTML(
  "beforeend",
  "<p>Hello</p>"
);
```

Positions:

```
beforebegin
afterbegin
beforeend
afterend
```

---

17. Removing Elements

Suppose:

```
<p id="msg">Hello</p>
```

Remove it:

```
document.getElementById("msg").remove();
```

Output:

```
<!-- gone -->
```

---

18. Removing Child Elements

```
parent.removeChild(child);
```

Example:

```
ul.removeChild(ul.firstElementChild);
```

Removes first list item.

---

19. Replacing Elements

HTML:

```
<p id="old">Old Text</p>
const oldP = document.getElementById("old");

const newP = document.createElement("p");

newP.textContent = "New Text";

oldP.replaceWith(newP);
```

Output:

```
<p>New Text</p>
```

---

20. Real-World Example: Todo List

HTML:

```
<input id="task">
<button id="addBtn">Add</button>

<ul id="list"></ul>
```

JavaScript:

```
const input = document.getElementById("task");
const btn = document.getElementById("addBtn");
const list = document.getElementById("list");

btn.addEventListener("click", () => {
```

```
  const li = document.createElement("li");

  li.textContent = input.value;

  list.appendChild(li);

  input.value = "";
});
```

If user enters:

```
Learn JavaScript
```

Output:

```
<ul>
  <li>Learn JavaScript</li>
</ul>
```

The DOM is updated dynamically.

---

21. Todo List with Delete Button

```
btn.addEventListener("click", () => {

  const li = document.createElement("li");

  li.textContent = input.value;

  const deleteBtn =
      document.createElement("button");

  deleteBtn.textContent = "Delete";

  deleteBtn.addEventListener("click", () => {
      li.remove();
  });

  li.appendChild(deleteBtn);

  list.appendChild(li);
});
```

Now each task gets its own delete button.

---

22. Difference Between HTML and DOM

HTML:

```
<h1>Hello</h1>
```

is just markup.

DOM:

```
document.querySelector("h1")
```

is the JavaScript object representation of that HTML.

Example:

```
const heading =
  document.querySelector("h1");

console.log(typeof heading);
```

Output:

```
object
```

Browser converts HTML into objects.

Common DOM Manipulation Methods to Remember

Method	Purpose
getElementById()	Find by ID
querySelector()	First matching element
querySelectorAll()	All matching elements
createElement()	Create new element
appendChild()	Add child
append()	Add child/text
insertBefore()	Insert before
remove()	Delete element
removeChild()	Delete child
replaceWith()	Replace element
textContent	Change text
innerHTML	Change HTML

Interview Questions

1. What is the DOM?

The DOM (Document Object Model) is a tree-like representation of an HTML document that allows JavaScript to access and manipulate webpage elements.

---

2. What is DOM Traversal?

DOM Traversal is moving through the DOM tree using relationships such as parent, child, and sibling nodes.

---

3. Difference between `children` and `childNodes`?

`children`

Returns only element nodes.

`childNodes`

Returns all nodes:

- Elements
- Text
- Comments

---

4. Difference between `append()` and `appendChild()`?

append()	appendChild()
Multiple items	One item
Text allowed	No
Modern	Older

---

5. How do you create a new DOM element?

```
const div = document.createElement("div");
```

---

6. How do you remove an element?

```
element.remove();
```

Quick Revision

```
// Find element
const box = document.querySelector(".box");

// Traversal
box.parentNode;
box.children;
box.firstElementChild;
box.lastElementChild;
box.nextElementSibling;
box.previousElementSibling;

// Create
const p = document.createElement("p");
p.textContent = "Hello";

// Add
document.body.appendChild(p);

// Remove
p.remove();

// Replace
oldElement.replaceWith(newElement);
```

Memory Trick:

DOM = **Tree of Nodes**

- Move around the tree → Traversal
- Add branches → `createElement()`, `appendChild()`
- Remove branches → `remove()`, `removeChild()`
- Replace branches → `replaceWith()`

If you master these operations, you'll be able to build dynamic features like Todo Apps, dashboards, modals, dropdowns, chat apps, and most interactive MERN frontend projects.

Event Bubbling, Capturing, and Event Delegation in JavaScript

These are some of the **most important JavaScript concepts** asked in interviews because they explain **how events (clicks, keyboard input, mouse movement, etc.) travel through the DOM**.

We'll learn them in this order:

1. What is an Event?
2. Understanding the DOM Tree
3. Event Propagation
4. Event Bubbling
5. Event Capturing
6. Bubbling vs Capturing
7. stopPropagation()
8. stopImmediatePropagation()
9. Event Delegation
10. Why Event Delegation is Powerful
11. Real-world Examples
12. Interview Questions
13. Practice Exercises

---

1. What is an Event?

An **event** is simply something that happens in the browser.

Examples:

- Clicking a button
- Pressing a key
- Scrolling
- Hovering over an element
- Submitting a form

Example:

```
<button id="btn">Click Me</button>
const btn = document.getElementById("btn");

btn.addEventListener("click", function () {
    console.log("Button clicked");
```

```
});
```

Output

```
Button clicked
```

When you click the button, the browser creates an **Event Object**.

---

2. Understanding the DOM Tree

Suppose we have:

```
<html>
    <body>
        <div id="parent">
            <button id="child">
                Click
            </button>
        </div>
    </body>
</html>
```

DOM Tree

```
HTML
 |
BODY
 |
DIV (parent)
 |
BUTTON (child)
```

When you click the button...

The event does **not** stay only on the button.

It **travels through the DOM Tree**.

This process is called

Event Propagation

---

3. Event Propagation

Whenever an event occurs, it goes through **three phases**.

```
1. Capturing Phase
        ↓

2. Target Phase
        ↓

3. Bubbling Phase
```

Imagine a package delivery.

```
HTML
↓

BODY
↓

DIV
↓

BUTTON

(Target reached)

BUTTON
↑

DIV
↑

BODY
↑

HTML
```

The event first goes **down**.

Then reaches the target.

Then comes **back up**.

Event Flow

```
HTML
 ↓
BODY
 ↓
DIV
 ↓
```

```
BUTTON

(Target)

BUTTON
 ↑
DIV
 ↑
BODY
 ↑
HTML
```

Downward = Capturing

Upward = Bubbling

---

4. Event Bubbling

This is the **default behavior**.

When an event happens,

It starts from the target element

then moves upward.

Example

```
<div id="parent">
    <button id="child">Click</button>
</div>
const parent = document.getElementById("parent");
const child = document.getElementById("child");

parent.addEventListener("click", function () {
    console.log("Parent clicked");
});

child.addEventListener("click", function () {
    console.log("Child clicked");
});
```

Click the button.

Output

```
Child clicked
Parent clicked
```

Why?

Because

```
BUTTON
↓

Target

↓

Bubble Up

BUTTON
↑
DIV
```

The event first happens on the button

then bubbles to its parent.

Bigger Example

```
<div id="grandparent">
    <div id="parent">
        <button id="child">
            Click
        </button>
    </div>
</div>
```

JavaScript

```
grandparent.addEventListener("click", () => {
    console.log("Grandparent");
});

parent.addEventListener("click", () => {
    console.log("Parent");
});

child.addEventListener("click", () => {
    console.log("Child");
});
```

Click button

Output

```
Child
Parent
```

```
Grandparent
```

Event bubbles upward.

Visual

```
Click Button

Child
 ↑

Parent
 ↑

Grandparent
```

---

5. Event Capturing

Capturing is the opposite.

Instead of moving upward,

the event travels from

```
HTML

↓

BODY

↓

DIV

↓

BUTTON
```

By default,

JavaScript **doesn't use capturing**.

You enable it by passing

```
true
```

or

```
{ capture: true }
```

Example

```
parent.addEventListener(
    "click",
    () => {
        console.log("Parent");
    },
    true
);

child.addEventListener(
    "click",
    () => {
        console.log("Child");
    },
    true
);
```

Click child.

Output

```
Parent
Child
```

Why?

Because the event goes downward.

```
Parent

↓

Child
```

Capturing vs Bubbling

Bubbling

```
Child

↑

Parent

↑
```

```
Grandparent
```

Capturing

```
Grandparent

↓

Parent

↓

Child
```

Easy trick to remember

```
Capture

TOP → DOWN

Bubble

BOTTOM → UP
```

---

6. Mixing Both

```
parent.addEventListener(
    "click",
    () => console.log("Parent Capture"),
    true
);

parent.addEventListener("click", () => console.log("Parent Bubble"));

child.addEventListener("click", () => console.log("Child"));
```

Click child.

Output

```
Parent Capture

Child

Parent Bubble
```

Flow

```
Capture
```

```
↓

Parent

↓

Child

Bubble

↑

Parent
```

---

7. stopPropagation()

Sometimes we don't want the event to bubble.

Example

```
child.addEventListener("click", function (event) {
    event.stopPropagation();

    console.log("Child");
});

parent.addEventListener("click", () => {
    console.log("Parent");
});
```

Click child.

Output

```
Child
```

Parent never executes.

Without stopPropagation()

```
Child

Parent
```

With stopPropagation()

```
Child
```

Real-Life Example

Suppose you have

```
<div class="card">
    <button>Delete</button>
</div>
```

Clicking card

```
Open Details
```

Clicking delete

```
Delete Item
```

Without stopPropagation()

Delete button

↓

Card click also happens

```
Delete

Open Details
```

Wrong!

Instead

```
deleteBtn.addEventListener("click", function (e) {

    e.stopPropagation();

    deleteItem();
});
```

Now

```
Delete

Only
```

---

8. stopImmediatePropagation()

Suppose one element has multiple listeners.

```
button.addEventListener("click", function (e) {

    e.stopImmediatePropagation();

    console.log("First");
});

button.addEventListener("click", function () {

    console.log("Second");
});
```

Output

```
First
```

Second never executes.

Difference

```
stopPropagation()

Stops parent listeners

BUT

Same element listeners still run.
stopImmediatePropagation()

Stops

Parent listeners

AND

Remaining listeners on same element.
```

---

9. Event Delegation

This is one of the **most important interview topics**.

Instead of adding listeners to every child,

we add ONE listener to the parent.

Why?

Because of **event bubbling**.

Suppose

```
<ul>
<li>Apple</li>
<li>Mango</li>
<li>Orange</li>
</ul>
```

Wrong approach

```
const items = document.querySelectorAll("li");
items.forEach(item => {
    item.addEventListener("click", () => {
        console.log(item.innerText);
    });
});
```

Every item gets a listener.

1000 items

↓

1000 listeners

Memory increases.

Better Approach

```
const ul = document.querySelector("ul");
ul.addEventListener("click", function (e) {
    console.log(e.target.innerText);
});
```

Only ONE listener.

Click Mango

Output

```
Mango
```

How?

Because

```
LI
↑
UL
```

The click bubbles to UL.

What is e.target?

When using event delegation:

```
ul.addEventListener("click", function (e) {
    console.log(e.target);
});
```

Click

```
<li>Apple</li>
```

Output

```
<li>Apple</li>
```

```
event.target
```

means

the actual element that triggered the event.

---

event.currentTarget

Example

```
ul.addEventListener("click", function (e) {
    console.log(e.target);
    console.log(e.currentTarget);
});
```

Output

```
target
<li>
currentTarget
<ul>
```

Difference

```
target
Where click happened.
currentTarget
Where listener is attached.
```

---

10. Why Event Delegation is Powerful

Suppose

```
for (let i = 0; i < 10000; i++) {
}
```

Imagine creating

```
10,000 Buttons
```

Without delegation

```
10,000 listeners
```

With delegation

`ONE listener`

Advantages

✅Less memory

✅Better performance

✅Cleaner code

✅Handles dynamically added elements

Dynamic Elements Example

HTML

```
<ul id="list">
</ul>
```

JavaScript

```
const list = document.getElementById("list");

list.addEventListener("click", function (e) {
    if (e.target.tagName === "LI") {
        console.log(e.target.textContent);
    }
});

const li = document.createElement("li");
li.textContent = "New Item";
list.appendChild(li);
```

Even though the `<li>` was created **after** the event listener was attached, clicking it still works because the click bubbles up to the `<ul>`, where the listener is waiting.

---

11. Real-World Examples

Example 1: Todo App

Instead of

```
deleteBtn.addEventListener(...)
```

for every todo,

Use

```
todoList.addEventListener("click", function (e) {
    if (e.target.classList.contains("delete")) {
        deleteTodo();
    }
});
```

Example 2: Table Rows

```
table.addEventListener("click", function (e) {
    if (e.target.tagName === "TD") {
        console.log(e.target.textContent);
    }
});
```

One listener.

Unlimited rows.

Example 3: Menu

```
menu.addEventListener("click", function (e) {
    if (e.target.matches(".menu-item")) {
        console.log(e.target.textContent);
    }
});
```

---

12. Interview Questions

Q1. What is event bubbling?

Answer:

Event bubbling is the default behavior in which an event starts from the target element and then propagates upward through its ancestors.

Q2. What is event capturing?

Answer:

Event capturing is the phase where an event travels from the root of the DOM down to the target element before reaching it.

Q3. Difference between bubbling and capturing?

Bubbling	Capturing
Bottom → Up	Top → Down
Default behavior	Must enable using `{ capture: true }`
Most commonly used	Less commonly used

Q4. What is event delegation?

Answer:

Event delegation is a technique where a single event listener is attached to a parent element, and events from child elements are handled using event bubbling and `event.target`.

Q5. Why use event delegation?

- Better performance

- Uses less memory
- Cleaner code
- Automatically works for dynamically added elements

Q6. Difference between `event.target` and `event.currentTarget`?

`event.target`	`event.currentTarget`
Element where the event originated	Element that owns the event listener
Changes depending on where the click happened	Always the element with the listener

---

Q7. Difference between `stopPropagation()` and `stopImmediatePropagation()`?

`stopPropagation()`	`stopImmediatePropagation()`
Stops the event from reaching parent elements	Stops parent propagation **and** prevents any remaining listeners on the same element from running

---

13. Practice Exercises

Try building these small projects to reinforce the concepts:

1. **Nested Boxes:** Create three nested `<div>` elements (grandparent → parent → child) and log the order of events using bubbling and capturing.
2. **Custom Context Menu:** Prevent the browser's default right-click menu with `event.preventDefault()` and display your own menu.
3. **Dynamic Todo List:** Add and remove todo items using a single delegated click listener on the list container.
4. **Image Gallery:** Use event delegation to detect which thumbnail was clicked and display it as the main image.
5. **Shopping Cart:** Create a table of products where clicking a "Remove" button deletes a row using a single listener on the table body.

Quick Revision (Cheat Sheet)

Concept	Meaning
Event	Something that happens in the browser (click, keypress, scroll, etc.)
Event Propagation	The path an event follows through the DOM
Capturing	Event travels from the root down to the target
Target Phase	The event reaches the element that triggered it
Bubbling	Event travels from the target back up through its ancestors (default)
`event.target`	The element that actually triggered the event
`event.currentTarget`	The element on which the event listener is attached
`stopPropagation()`	Stops the event from moving to ancestor elements
`stopImmediatePropagation()`	Stops ancestor propagation and any remaining listeners on the same element
Event Delegation	Attach one listener to a parent and handle child events using bubbling

Memory trick:

```
Event Flow

CAPTURING
HTML
  ↓
BODY
  ↓
DIV
  ↓
BUTTON  ← Target
  ↑
DIV
  ↑
BODY
  ↑
HTML
BUBBLING
```

Remember:

- **Capture = Top → Down**

- **Bubble = Bottom → Up**
- **Delegation = One parent listener handles many children using `event.target`**

Browser Performance: Debouncing and Throttling (Write from Scratch)

These are two of the **most frequently asked JavaScript interview topics** because they are used to improve browser performance by controlling how often a function executes.

Examples:

- Search suggestions
- Button clicks
- Window resizing
- Scrolling
- Mouse movement
- API requests

Why do we need Debouncing and Throttling?

Imagine your browser is receiving hundreds of events every second.

For example:

```
window.addEventListener("scroll", () => {
    console.log("Scrolling...");
});
```

When you scroll for just **1 second**, the browser may fire the `scroll` event **100–200 times**.

If every event:

- Calls an API
- Updates the DOM
- Performs heavy calculations

then your website becomes slow.

That's where Debouncing and Throttling help.

Real-life Analogy

Imagine someone continuously rings your doorbell.

```
Ring Ring Ring Ring Ring Ring Ring
```

There are two ways to handle it.

Debouncing

You wait until they stop ringing.

```
Ring Ring Ring Ring Ring

(wait)

Open Door
```

Only one action happens.

Throttling

You decide:

I'll open the door only once every 5 seconds no matter how many times they ring.

```
Ring Ring Ring

Open Door

Ring Ring Ring

Open Door
```

Actions happen repeatedly, but with a fixed interval.

What is Debouncing?

Definition

Debouncing delays function execution until the event has stopped happening for a specified amount of time.

In simple words:

"Wait until the user finishes."

Example

Search Box

Suppose a user types

```
H
He
Hel
Hell
Hello
```

Without Debouncing:

```
API Call
API Call
API Call
API Call
API Call
```

Five API requests.

With Debouncing (500ms)

```
H
He
Hel
Hell
Hello

(wait 500ms)

API Call
```

Only one request.

Huge performance improvement.

Timeline

Without Debouncing

```
Typing

|H|He|Hel|Hell|Hello|

API API API API API
```

With Debouncing

```
Typing

|H|He|Hel|Hell|Hello|

........500ms......

API
```

How Debouncing Works

Every time the event occurs:

```
Clear previous timer

Start new timer

If timer completes

Execute function
```

So every new event restarts the waiting period.

Writing Debounce from Scratch

Step 1

Basic structure

```
function debounce(func, delay) {

}
```

Parameters

```
func → original function
```

```
delay → waiting time
```

Step 2

Store timer

```
function debounce(func, delay) {

    let timer;

}
```

Step 3

Return another function

```
function debounce(func, delay) {

    let timer;

    return function () {

    };

}
```

Why?

Because we don't execute immediately.

Instead we return a new controlled function.

Step 4

Clear previous timer

```
return function () {

    clearTimeout(timer);

}
```

If user types again

Old timer is removed.

Step 5

Create new timer

```
return function () {

    clearTimeout(timer);

    timer = setTimeout(() => {
        func();
    }, delay);

}
```

Now the function executes only after delay.

Final Debounce Function

```
function debounce(func, delay) {

    let timer;

    return function (...args) {

        clearTimeout(timer);

        timer = setTimeout(() => {
            func(...args);
        }, delay);

    };

}
```

Why use `...args`?

Suppose

```
function search(text) {
    console.log(text);
}
```

We don't know what arguments future functions may need.

So we forward all arguments.

```
func(...args);
```

Why do we use `clearTimeout()`?

Without it

```
Type A

Timer1

Type B

Timer2

Type C

Timer3
```

All timers execute.

With `clearTimeout()`

```
Type A

Timer1

Type B

Cancel Timer1

Timer2

Type C

Cancel Timer2

Timer3

Only Timer3 runs
```

Example

```
<input id="search">
function search(value) {
    console.log("Searching:", value);
}

const debouncedSearch = debounce(search, 500);

document
```

```
    .getElementById("search")
    .addEventListener("input", (e) => {
        debouncedSearch(e.target.value);
    });
```

Typing

```
J
Ja
Jav
Java
JavaS
JavaSc
```

Output

```
Searching: JavaSc
```

Only once.

Another Example

Window Resize

Without debounce

```
window.addEventListener("resize", () => {
    console.log("Resize");
});
```

Hundreds of logs.

With debounce

```
window.addEventListener(
    "resize",
    debounce(() => {
        console.log("Resize finished");
    }, 300)
);
```

Only once after resizing stops.

What is Throttling?

Definition

Throttle allows a function to execute **at most once every fixed interval**, no matter how many times the event fires.

In simple words:

"Run regularly, but not too often."

Example

Scrolling

Without throttle

```
Scroll

Function
Function
Function
Function
Function
Function
Function
```

With throttle (1 second)

```
Scroll

Function

(wait 1 sec)

Function

(wait 1 sec)

Function
```

Timeline

Without throttle

```
Scroll Scroll Scroll Scroll
```

```
Execute Execute Execute Execute
```

With throttle

```
Scroll Scroll Scroll Scroll

Execute

(wait)

Execute

(wait)

Execute
```

Writing Throttle from Scratch

Step 1

```
function throttle(func, delay) {

}
```

Step 2

Create a flag

```
let shouldWait = false;
```

Initially

```
false

Meaning

Function can execute
```

Step 3

Return function

```
return function (...args) {

}
```

Step 4

Check flag

```
if (shouldWait) return;
```

If already waiting

Ignore.

Step 5

Execute

```
func(...args);
```

Step 6

Lock

```
shouldWait = true;
```

Step 7

Unlock after delay

```
setTimeout(() => {
    shouldWait = false;
}, delay);
```

Final Throttle Function

```
function throttle(func, delay) {

    let shouldWait = false;

    return function (...args) {

        if (shouldWait) return;

        func(...args);

        shouldWait = true;
```

```
        setTimeout(() => {
            shouldWait = false;
        }, delay);

    };

}
```

Example

```
function handleScroll() {
    console.log("Scroll event");
}

window.addEventListener(
    "scroll",
    throttle(handleScroll, 1000)
);
```

Even if the browser fires 200 events,

Output

```
Scroll event

(after 1 second)

Scroll event

(after 1 second)

Scroll event
```

Another Throttle Example

Button Click

```
const btn = document.querySelector("button");

btn.addEventListener(
    "click",
    throttle(() => {
        console.log("Clicked");
    }, 2000)
);
```

Even if user clicks

```
Click Click Click Click Click
```

Output

```
Clicked

(after 2 sec)

Clicked
```

Debounce vs Throttle

Feature	Debounce	Throttle
Runs	After events stop	At fixed intervals
Waits	Yes	No
Misses intermediate events	Yes	No (executes periodically)
Best for	Search, resize	Scroll, mousemove, drag
API calls	Excellent	Usually not ideal
User typing	Perfect	Not recommended

Visual Comparison

Imagine typing

```
A B C D E
```

Debounce

```
A B C D E

(wait)

Run
```

Throttle (500ms)

```
A B
```

```
Run

C D

Run

E

Run
```

When to Use Which?

Use Debounce

- ✅Search suggestions
- ✅Form validation after typing
- ✅Auto-save after the user stops typing
- ✅Window resize
- ✅Filtering large lists

Use Throttle

- ✅Infinite scrolling
- ✅Scroll progress bar
- ✅Mouse movement tracking
- ✅Drag-and-drop updates
- ✅Game controls
- ✅Prevent repeated button actions

Interview Questions

1. What is Debouncing?

It delays the execution of a function until a specified time has passed since the last event. It helps avoid unnecessary repeated function calls.

---

2. What is Throttling?

It limits how often a function can execute by allowing it to run at most once during a fixed time interval.

---

3. Difference between Debounce and Throttle?

- **Debounce** waits until events stop before running the function.
- **Throttle** runs the function at regular intervals while events continue.

---

4. Why do they improve browser performance?

They reduce the number of expensive operations (like API calls, DOM updates, or calculations) triggered by high-frequency events, resulting in smoother performance and lower resource usage.

---

5. Which events commonly use Debounce?

- Search input
- Window resize
- Auto-save
- Form validation

---

6. Which events commonly use Throttle?

- Scroll
- Mouse move
- Drag-and-drop
- Infinite scroll
- Repeated button clicks

Memory Trick

Think of these phrases:

Debounce = "Wait until the user stops."

```
Typing...
Typing...
Typing...

(wait)

Run
```

Throttle = "Run every few seconds."

```
Scrolling...

Run

Scrolling...

Run

Scrolling...

Run
```

Keeping these simple rules in mind will help you quickly decide which technique to use in interviews and real-world applications.

Web Storage APIs: localStorage, sessionStorage, and Cookies

These are some of the most commonly asked topics in JavaScript interviews.

They all allow us to **store data in the browser**, but they work differently.

Think of your browser like a room.

- **Cookies** = Small sticky notes that are also sent to the server.
- **localStorage** = A permanent cupboard where you store things.
- **sessionStorage** = A temporary desk that gets cleaned when you leave the room.

Why do we need storage?

Imagine a shopping website.

A user:

- logs in
- changes theme to dark mode
- adds products to cart

If the page reloads, all this information would disappear unless we store it somewhere.

Browser storage allows us to save data.

Examples:

- Login information
- Theme preference
- Shopping cart
- User settings
- Recently viewed products

Browser Storage Overview

Storage	Capacity	Sent to Server?	Expires
localStorage	~5-10 MB	✗No	Never (until deleted)
sessionStorage	~5-10 MB	✗No	When tab closes
Cookies	~4 KB	☑Yes	Depends on expiry

---

localStorage

What is localStorage?

localStorage stores data **permanently** inside the browser.

It remains even after:

- Refresh
- Browser restart
- Computer restart

until someone removes it.

Think of it as:

A cupboard inside your room.

Anything you put inside stays there until you throw it away.

Example

User selects Dark Mode.

```
localStorage.setItem("theme", "dark");
```

Next day user opens website.

```
console.log(localStorage.getItem("theme"));
```

Output

```
dark
```

The browser remembered it.

Methods of localStorage

1. setItem()

Stores data.

```
localStorage.setItem("name", "John");
```

Storage becomes

```
name → John
```

---

2. getItem()

Reads data.

```
const name = localStorage.getItem("name");

console.log(name);
```

Output

```
John
```

---

3. removeItem()

Deletes one item.

```
localStorage.removeItem("name");
```

Now

```
name → deleted
```

---

4. clear()

Deletes everything.

```
localStorage.clear();
```

Everything stored disappears.

---

5. key()

Returns key by index.

```
localStorage.key(0);
```

Example output

```
theme
```

---

6. length

Number of stored items.

```
console.log(localStorage.length);
```

Example

Save username.

```
localStorage.setItem("username", "Alice");
```

```
console.log(localStorage.getItem("username"));
```

Output

```
Alice
```

Updating data

```
localStorage.setItem("username", "Bob");
```

Old value

```
Alice
```

New value

```
Bob
```

Storing Numbers

Everything becomes a string.

```
localStorage.setItem("age", 25);
```

Reading

```
console.log(localStorage.getItem("age"));
```

Output

```
"25"
```

Notice

It's a string.

Storing Objects

Objects cannot be stored directly.

Wrong

```
const user = {
    name: "John"
};

localStorage.setItem("user", user);
```

Stored value

```
[object Object]
```

Wrong!

Correct way

Convert object into JSON.

```
const user = {
    name: "John",
    age: 24
};

localStorage.setItem(
    "user",
    JSON.stringify(user)
);
```

Reading

```
const data = JSON.parse(
    localStorage.getItem("user")
);

console.log(data.name);
```

Output

```
John
```

---

localStorage Example

```
const themeBtn = document.getElementById("btn");

themeBtn.addEventListener("click", () => {

    localStorage.setItem("theme", "dark");

});
```

When page loads

```
const theme = localStorage.getItem("theme");

if(theme === "dark"){
    document.body.classList.add("dark");
}
```

The theme stays even after refresh.

---

sessionStorage

Very similar to localStorage.

Difference:

It survives only while the browser tab is open.

Think of it like

Notes on your study table.

When you leave the room (close tab),

everything is thrown away.

Example

```
sessionStorage.setItem("username", "Alice");
```

Refresh page

```
Still exists
```

Close browser tab

```
Gone
```

Methods

Exactly same.

```
sessionStorage.setItem()
```

```
sessionStorage.getItem()
```

```
sessionStorage.removeItem()
```

```
sessionStorage.clear()
```

Example

```
sessionStorage.setItem("page", "checkout");
```

Refresh

```
checkout
```

Close tab

```
Deleted
```

When to use sessionStorage?

Examples

- Multi-step forms
- Temporary shopping data
- OTP verification state
- Temporary filters
- Wizard forms

Cookies

Cookies are much older than localStorage.

Main purpose:

Store small pieces of information.

Unlike localStorage,

Cookies are automatically sent to the server with every HTTP request (for matching domains and paths).

Imagine

You visit

```
amazon.com
```

Browser sends

```
Cookie:
session=ABC123
```

Server immediately knows

```
User is logged in.
```

Cookie Example

```
document.cookie = "username=John";
```

Read

```
console.log(document.cookie);
```

Output

```
username=John
```

Multiple Cookies

```
document.cookie = "username=John";
document.cookie = "theme=dark";
```

Reading

```
console.log(document.cookie);
```

Output

```
username=John; theme=dark
```

Cookie Expiry

Without expiry

```
document.cookie = "user=John";
```

This becomes a session cookie.

It disappears when browser closes.

Set expiry

```
document.cookie =
"username=John; expires=Fri, 31 Dec 2030 12:00:00 UTC";
```

Now cookie stays until 2030.

Cookie Path

```
document.cookie =
"theme=dark; path=/";
```

Meaning

Entire website can access it.

Delete Cookie

Set expiry in past.

```
document.cookie =
"username=; expires=Thu, 01 Jan 1970 00:00:00 UTC";
```

Deleted.

Why Cookies are Important?

Cookies are mostly used by servers.

Examples

- Login session
- Authentication
- User identification
- Tracking
- Analytics

Comparison

Persistence

localStorage

Saved forever
sessionStorage

Until tab closes
Cookies

Depends on expiry

Size

Cookies

Around 4 KB
localStorage

Around 5 MB
sessionStorage

Around 5 MB

Sent to Server

Cookies

YES

localStorage

NO

sessionStorage

NO

Which one is faster?

Generally

`localStorage`

and

`sessionStorage`

are faster because they stay only in the browser.

Cookies travel with every HTTP request.

Practical Examples

Example 1

Remember Dark Mode

`Use localStorage`

Because

`User expects it forever.`

Example 2

Current Form Step

`Use sessionStorage`

Because

`Temporary information.`

Example 3

Login Session

`Use Cookies`

Because

`Server needs to know who the user is.`

Example 4

Shopping Cart

Usually

`localStorage`

if user isn't logged in.

Database + Cookies

if user is logged in.

---

localStorage vs sessionStorage

Feature	localStorage	sessionStorage
Refresh	✅Yes	✅Yes
Close Tab	✅Still exists	❌Deleted
New Tab	✅Shared across tabs of the same origin	❌Separate for each tab
Capacity	5–10 MB	5–10 MB

---

localStorage vs Cookies

Feature	localStorage	Cookies
Capacity	5–10 MB	4 KB
Sent to Server	❌No	✅Yes
Expiry	Never (unless removed)	Configurable
Performance	Faster	Slower (extra network overhead)

Feature	localStorage	Cookies
Best Use	Preferences, UI state	Authentication, sessions

Security Considerations

Never store sensitive information in localStorage

Avoid storing:

- Passwords
- Credit card numbers
- API secrets
- Private keys

Why? JavaScript running on the page can access `localStorage`. If your site has an `XSS (Cross-Site Scripting)` vulnerability, malicious scripts could read that data.

✕Bad:

```
localStorage.setItem("password", "mySecret123");
```

Cookies can be more secure

When authentication cookies are set by the server with flags like:

- `HttpOnly` (JavaScript cannot read the cookie)
- `Secure` (only sent over HTTPS)
- `SameSite` (helps protect against CSRF attacks)

they are generally a safer place for session tokens than `localStorage`.

Common Interview Questions

1. What is the difference between localStorage and sessionStorage?

- `localStorage` persists until manually removed.
- `sessionStorage` is cleared when the browser tab is closed.

- Both store data only in the browser and are **not** automatically sent to the server.

---

2. Can localStorage store objects?

No. It only stores strings.

Use:

```
localStorage.setItem("user", JSON.stringify(user));
```

Read back with:

```
const user = JSON.parse(localStorage.getItem("user"));
```

---

3. Why are cookies still used if localStorage exists?

Because cookies are automatically included in HTTP requests, allowing the server to identify users, maintain sessions, and handle authentication. `localStorage` stays entirely on the client.

---

4. Which storage is best for authentication tokens?

For most web applications, **secure, `HttpOnly` cookies** are preferred for session or authentication tokens because they cannot be accessed by client-side JavaScript, reducing the risk from XSS attacks.

---

5. What are the storage limits?

- **Cookies:** ~4 KB per cookie
- **localStorage:** Typically 5–10 MB (browser-dependent)
- **sessionStorage:** Typically 5–10 MB (browser-dependent)

Real-World Decision Guide

Scenario	Best Choice	Reason

Scenario	Best Choice	Reason
Dark mode preference	`localStorage`	Keep preference across visits
Language selection	`localStorage`	User expects it to persist
Multi-step form progress	`sessionStorage`	Temporary for the current tab
Temporary search filters	`sessionStorage`	No need after closing the tab
User login session	Secure `HttpOnly` cookie	Server needs it and JavaScript can't access it
Shopping cart (guest user)	`localStorage`	Preserve cart across browser restarts
Analytics or tracking	Cookies	Included with requests when appropriate

---

Quick Revision (1-Minute Recap)

- **`localStorage`**
  - Stores **strings only**
  - Persists until manually deleted
  - Shared across tabs of the same website
  - Best for user preferences and persistent client-side data
- **`sessionStorage`**
  - Stores **strings only**
  - Cleared when the tab is closed
  - Separate storage for each tab
  - Best for temporary session-specific data
- **Cookies**
  - Small (about 4 KB)
  - Automatically sent with HTTP requests
  - Can have expiration dates
  - Best for authentication, sessions, and server-related state
  - More secure for session tokens when configured with `HttpOnly`, `Secure`, and `SameSite`

Understanding **when** to use each storage mechanism is often more important in interviews than simply memorizing their differences.

This is an excellent project because it combines many important JavaScript concepts that companies ask in interviews:

- ✅DOM Manipulation
- ✅Fetch API
- ✅Async/Await
- ✅Event Delegation
- ✅Debouncing
- ✅Infinite Scrolling
- ✅IntersectionObserver
- ✅Browser Performance
- ✅Responsive CSS
- ✅Error Handling

Let's build it **step by step** in a beginner-friendly way.

Project Overview

We are building an **Infinite Scroll Image Gallery**.

The application will:

- Load images from an API
- Display them in a responsive grid
- Automatically load more images when the user reaches the bottom
- Use **IntersectionObserver** instead of listening to every scroll event
- Use **custom debouncing**
- Use **event delegation** for clicking images

The final result will look similar to:

```
----------------------------------------
 Search Images

□   □   □

□   □   □

□   □   □

.............

(scroll)

Automatically loads more images
```

....

Step 1: Folder Structure

```
Infinite-Gallery/

├── index.html
├── style.css
├── script.js
```

Step 2: HTML

```
<!DOCTYPE html>
<html>

<head>
    <title>Infinite Gallery</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

<h1>Infinite Scroll Gallery</h1>

<div id="gallery"></div>

<div id="loader">
    Loading...
</div>

<div id="observer"></div>

<script src="script.js"></script>

</body>

</html>
```

Notice the empty

```
<div id="observer"></div>
```

This is the element that IntersectionObserver watches.

Step 3: CSS

```
body{
    margin:20px;
    font-family:Arial;
    background:#f4f4f4;
}

h1{
    text-align:center;
}

#gallery{
    display:grid;

    grid-template-columns:
        repeat(auto-fit,minmax(250px,1fr));

    gap:20px;
}

.card{
    overflow:hidden;
    border-radius:10px;
    background:white;
    box-shadow:0 5px 10px rgba(0,0,0,.2);
}

.card img{
    width:100%;
    display:block;
}

#loader{
    text-align:center;
    margin:30px;
}
```

Output:

```
+--------+
| Image  |
+--------+

+--------+
| Image  |
+--------+
```

Step 4: Fetch Images

We'll use

```
https://picsum.photos
```

Example

```
https://picsum.photos/300/200
```

returns a random image.

JavaScript

```
const gallery = document.getElementById("gallery");
```

Function

```
function createImage(){
    const div=document.createElement("div");
    div.className="card";
    const img=document.createElement("img");
    img.src=`https://picsum.photos/300/200?random=${Math.random()}`;
    div.appendChild(img);
    gallery.appendChild(div);
}
```

Load first 12 images

```
for(let i=0;i<12;i++){
    createImage();
}
```

Result

```
Image
Image
Image
Image
```

Step 5: Infinite Scroll

Old way

```
window.addEventListener("scroll",...)
```

Problem:

```
Scroll fires hundreds of times.
```

Example

```
User scrolls 1 second
```

↓

```
200 events
```

Bad for performance.

Instead we'll use

```
IntersectionObserver
```

Step 6: IntersectionObserver

Observe the bottom element.

```
const observer=document.getElementById("observer");
```

Create observer

```
const io=new IntersectionObserver((entries)=>{
    if(entries[0].isIntersecting){
        loadImages();
    }
});
```

Start observing

```
io.observe(observer);
```

Whenever the bottom becomes visible

↓

```
loadImages()

↓

More images appear

↓

Bottom moves down

↓

Again observed

↓

More images
```

Amazing performance.

Step 7: Create loadImages()

```
function loadImages(){

    for(let i=0;i<9;i++){

        createImage();

    }

}
```

Done.

Now scrolling keeps loading forever.

Step 8: Add Loading Effect

Before loading

```
loader.style.display="block";
```

After loading

```
loader.style.display="none";
```

Example

```
function loadImages(){
loader.style.display="block";
setTimeout(()=>{
for(let i=0;i<9;i++){
createImage();
}
loader.style.display="none";
},1000);
}
```

Step 9: Custom Debounce

Even IntersectionObserver may trigger repeatedly.

Let's debounce.

Custom implementation

```
function debounce(fn, delay){
    let timer;
    return function(){
        clearTimeout(timer);
        timer=setTimeout(()=>{
            fn();
        },delay);
    }
}
```

Now

```
const debouncedLoad=debounce(loadImages,300);
```

Observer

```
const io=new IntersectionObserver((entries)=>{

if(entries[0].isIntersecting){

debouncedLoad();

}

});
```

Now rapid observer callbacks become

```
Call
Call
Call
Call
Call

↓

Only ONE executes
```

Step 10: Event Delegation

Suppose clicking an image should enlarge it.

Bad approach

```
document.querySelectorAll("img").forEach(img=>{

img.addEventListener(...)

});
```

If 500 images exist

```
500 listeners
```

Instead

```
gallery.addEventListener("click",(e)=>{

if(e.target.tagName==="IMG"){

alert("Image Clicked");

}
```

```
});
```

Only

```
ONE
```

```
listener
```

for

```
500+
```

```
images
```

This is Event Delegation.

How Event Delegation Works

Imagine

```
Gallery
```

```
|
```

```
├── Image
```

```
├── Image
```

```
├── Image
```

```
├── Image
```

Click

```
Image
```

```
↓
```

```
event bubbles
```

```
↓
```

```
Gallery
```

```
↓
```

```
Gallery checks
```

```
Was it an image?
```

```
↓
Yes
↓
Perform action
```

Only one listener.

Project Flow

```
Page Loads
↓
Load Images
↓
Display Grid
↓
User Scrolls
↓
Bottom Visible
↓
IntersectionObserver Fires
↓
Debounce
↓
Load More Images
↓
Append Images
↓
Repeat Forever
```

Bonus Improvements

Lazy Loading

```
img.loading = "lazy";
```

Fade Animation

```
.card{

animation:fade .4s;

}

@keyframes fade{

from{

opacity:0;

transform:translateY(20px);

}

to{

opacity:1;

transform:translateY(0);

}

}
```

Responsive Grid

Already handled with

```
grid-template-columns:
repeat(auto-fit,minmax(250px,1fr));
```

Image Modal

Click image

↓

Open full-size preview.

Search Feature

Use an API like:

- Unsplash
- Pixabay
- Pexels

Complete Concepts Used

Concept	Used In Project
DOM Manipulation	Creating image cards
createElement()	Creating images dynamically
appendChild()	Adding cards to the page
Async Loading	Fetching or generating images
CSS Grid	Responsive layout
Event Delegation	One click listener for all images
Debouncing	Prevent multiple rapid loads
IntersectionObserver	Detecting when to load more
Infinite Scroll	Automatically loading additional images
Lazy Loading	Faster page performance

Interview Questions

1. What is Event Delegation?

Definition:

Event delegation is a technique where you attach a single event listener to a parent element instead of adding listeners to each child element. It works because of **event bubbling**—events triggered on child elements bubble up to their parent.

Example:

Instead of:

```
document.querySelectorAll(".btn").forEach(button => {
  button.addEventListener("click", () => {
    console.log("Button clicked");
  });
});
```

Use one listener on the parent:

```
const container = document.getElementById("container");

container.addEventListener("click", (event) => {
  if (event.target.classList.contains("btn")) {
    console.log("Button clicked");
  }
});
```

Here:

- The click happens on a button.
- The event bubbles to `container`.
- `event.target` tells us which child was actually clicked.

---

2. Why is Event Delegation Useful for Performance?

It improves performance because:

- **Fewer event listeners:** One listener can handle hundreds or thousands of child elements.
- **Lower memory usage:** Each event listener consumes memory.
- **Works with dynamically added elements:** New child elements automatically work without adding new listeners.
- **Simpler code:** You manage events in one place instead of attaching/removing listeners repeatedly.

Example:

If you have **1,000 images** in an infinite-scroll gallery:

- Without delegation: **1,000 click listeners**
- With delegation: **1 click listener** on the gallery container

This is more memory-efficient and easier to maintain.

Common Interview Follow-up Questions

Q1. Does event delegation work for every event?

No. It works best for events that **bubble**, such as:

- `click`
- `keydown`
- `keyup`
- `change`

Some events like `focus` and `blur` do **not** bubble (though `focusin` and `focusout` do).

Q2. What is `event.target` vs `event.currentTarget`?

```
gallery.addEventListener("click", (event) => {
  console.log(event.target);        // The actual clicked element (e.g., an
<img>)
  console.log(event.currentTarget); // The element with the event listener
(gallery)
});
```

- `event.target`: The element where the event originated.
- `event.currentTarget`: The element whose listener is currently executing.

Q3. When should you use `IntersectionObserver` instead of the `scroll` event?

Use `IntersectionObserver` when you need to know **when an element enters or leaves the viewport**, such as:

- Infinite scrolling
- Lazy-loading images
- Triggering animations

It is generally more efficient than listening to every `scroll` event because the browser optimizes the observation internally.